

Estou desenvolvendo um projeto pessoal chamado provisoriamente Azor.

Sou programador júnior, estou aprendendo desenvolvimento web moderno e quero construir o projeto de forma gradual, com explicações simples e decisões técnicas bem justificadas.

O Azor é um Sistema de Gestão Financeira Web que futuramente pode evoluir para um ERP pessoal. O objetivo principal é aprender em público, construir portfólio e aprofundar conhecimentos em desenvolvimento web, arquitetura de software, regras de negócio financeiras e práticas usadas em sistemas corporativos.

Stack definida:

- Backend: C# 14, .NET 10, ASP.NET Core Web API
- Banco de dados: SQL Server
- ORM: Entity Framework Core
- Frontend: Angular
- UI: Angular Material
- Editor: VS Code

Arquitetura do backend:

- Monólito modular com Clean Architecture leve.
- Azor.Domain: entidades, enums, value objects e regras centrais do negócio.
- Azor.Application: casos de uso, DTOs, validações e interfaces.
- Azor.Infrastructure: Entity Framework Core, SQL Server, DbContext, repositórios e serviços externos.
- Azor.Api: controllers/endpoints, configuração da aplicação, CORS, injeção de dependência e comunicação HTTP.

A dependência entre projetos deve seguir esta ideia:

- Domain não depende de ninguém.
- Application depende de Domain.
- Infrastructure depende de Application e Domain.
- Api depende de Application e Infrastructure.

Frontend Angular:

- Organizado por funcionalidades.
- Estrutura prevista:
 - core: serviços globais, interceptors, layout.
 - shared: componentes reutilizáveis, models, pipes.
 - features: páginas e funcionalidades do sistema.
- Primeiras features:
 - home
 - auth/login
 - auth/register
 - dashboard
 - financial-accounts
 - categories
 - transactions

Escopo inicial do MVP:

- Homepage apresentando o projeto.
- Botões para cadastro e login.
- Endpoint simples no backend para testar se a API está online.
- Comunicação Angular ↔ ASP.NET Core funcionando.
- Depois disso, evoluir gradualmente para cadastro de usuário, login, contas financeiras, categorias, lançamentos, receitas, despesas, transferências, saldo por conta, saldo consolidado e dashboard simples.

Decisões importantes:

- Não usar microserviços no início.
- Não usar CQRS pesado.
- Não usar mensageria.
- Não usar Redis/cache inicialmente.
- Não usar abstrações excessivas.
- Priorizar código simples, legível e incremental.
- Explicar sempre: o que é, por que usar, o que evitar e como aplicar de forma simples.